

DATASTORE_KEY : from TRNG to the CMAC on your data

The one-sentence answer

`DATASTORE_KEY` is **not derived from anything** — it is 16 bytes of on-chip TRNG entropy, wrapped in a SHE M1|M2|M3 message (whose K1–K4 are the only derived keys involved), written into slot `KEY_3` with `KU|WP` flags, and from then on used *exclusively* as a CMAC key to seal 64-byte storage blocks and NvM-managed data blocks.

VOCABULARY PRECISION (THIS TRIPS PEOPLE UP)

Generated ≠ **derived**. The key *value* in `KEY_3` is pure random — fresh on every ECU, unknowable offline. What *is* derived (via the SHE KDF) are the transport keys K1–K4 that encrypt and authenticate the key during registration. Deriving a slot key from a device secret (like `SECRET_KEY`) is a different design pattern — this project deliberately doesn't use it.

1 • Provisioning: the journey into KEY_3

It happens as step 2 of `SecSB_EnableSecurity` (`secure_boot.c:145`), only once secure boot itself is enabled or already write-protected:

GENERATE · `SECURE_BOOT.C:433` → `CRYPTO_LIBRARY.C:182`

`SecSB_GenerateRamKey` — ICUSE TRNG produces 16 bytes; staged into the volatile `RAM_KEY` as a plaintext key (`CMD_LOAD_PLAIN_KEY`).



DESCRIBE · `SECURE_BOOT.C:437–447`

`keyData` : `keyId` = `DATASTORE_KEY` (→ `KEY_3` → `0x06`), `authKeyId` = **same key** (self-authenticated first load, `authKey` = `0128` per manual §4.4), `UID` = wildcard, `counter` = **1**, `flags` = `KU | WP`, `key` = the 16 random bytes.



WRAP · `KEY_MANAGER.C:99` → `CRYPTO_LIBRARY.C:456`

`SecLib_GenerateM1M2M3` + `SecLib_KeySHEUpdate` — builds and ships the protected message to the driver.



COMMIT · `CRY_30_RH850ICUS_HW.C:1100`

`CMD_LOAD_KEY` — ICUSE decrypts, verifies and writes slot `KEY_3` in the ICU region of the data flash. The value is now unreadable from outside, forever.



PROVE · `SECURE_BOOT.C:463–480`

A CMAC generate + verify round-trip over a fixed 32-byte pattern (`pcMACdata`, `secure_boot.c:85`) confirms the slot is live. `WP` then locks it permanently.

The anatomy of M2 — where the key actually hides

M2 is the only place the plaintext key value ever travels, and it never travels as plaintext. Built byte-by-byte in `SecLib_GenerateM2` (`key_manager.c:197–246`):

BYTES	CONTENT	SOURCE
0–3	<code>counter<<4 flags[7:4]</code> (big-endian)	<code>counter</code> = 1, <code>flags</code> = <code>KU WP</code> → <code>0x00 00 00 19</code> (<code>0x10 0x09</code>)
4	<code>flags[3:0] << 4</code> (plus padding)	upper flag nibble
5–15	zero padding (94 zero bits total)	SHE-mandated
16–31	the new 16-byte key	the TRNG bytes from <code>RAM_KEY</code>

This 32-byte block is AES-CBC encrypted with `K1 = KDF(KEY_AuthID, KEY_UPDATE_ENC_C)` (IV = `0`). Note the neat implementation trick: since

AuthID = ID and the slot is empty, KEY_AuthID behaves as all-zeros — so K1/K2 are derived from the **all-zero auth key** (key_manager.c:233, 285), loaded temporarily into RAM_KEY to run the AES/CMAC primitives. ICUSE then unwraps M2 inside the secure unit, where the value never re-appears on any bus.

2 · The job: sealing data at rest

Once provisioned, DATASTORE_KEY appears in exactly two protection mechanisms — both CMAC-only, as KU dictates.

Mechanism A — the hsto_ secure API (data_protection.c)

A secure block is exactly BLOCK_SIZE = 64 bytes (crypto_library.h:49) with a fixed layout:

BYTES 0–15	CMAC — computed over the payload, stored by the writer
BYTES 16–63	payload — stored in plaintext (integrity, not confidentiality)

Write — hsto_WriteSecure (data_protection.c:106): CMAC over data[16..63] generated with DATASTORE_KEY and written into data[0..15] ; the 64-byte block then goes to the master location (flash address or NvM block). With FLS_DRIVER , a failed master write falls back to a backup block at target + BLOCK_SIZE .

Read — hsto_ReadSecure (data_protection.c:22): CMAC over data[16..63] recomputed and compared against the stored data[0..15] . On mismatch: pre-secure-boot builds retry the backup block; once secure boot is enabled the payload is **zeroized and an error returned** (data_protection.c:84). The consumer never sees unverified data.

Mechanism B — NvM callbacks (nvm_driver.c)

For AUTOSAR NvM-managed blocks, the same key works through the standard hook points — no caller changes needed:

HOOK	FUNCTION	WHEN
Write callback	SecLib_NVM_MACVerify_Write (nvm_driver.c:541)	Computes the CMAC with DATASTORE_KEY over the block's data region and embeds it before the write
PostReadTransformation	SecLib_NVM_MACVerify_Read (nvm_driver.c:596)	Recomputes and verifies; on failure returns E_NOT_OK so NvM reports the block as INTEGRITY_FAILED

Protected block families in this codebase: SAM (MAC || data), Dataset (two security-log entries, two CMACs each), and ODOM_01...10 odometer blocks in the CLI variant. All share the one DATASTORE_KEY.

3 · What this buys you — and what it doesn't

PROPERTY	COVERED?	WHY
Integrity (tamper detection)	✓	Any bit flipped in payload or MAC → CMAC verify fails → block zeroized / INTEGRITY_FAILED
Authenticity (origin)	✓	Forging a valid CMAC requires the key inside KEY_3, which cannot leave ICUSE and is CMAC-only (KU), frozen (WP)
Confidentiality	✗	Payload is stored in the clear; CMAC seals, it does not encrypt
Freshness / anti-rollback	✗	An attacker with old flash copies can replay a whole old-but-valid block; nothing in the MAC binds it to "now"

ATTACKER'S DILEMMA

To tamper with protected data, an attacker must produce a new valid CMAC — which requires the key. The key lives in an unreadable ICU-region slot, restricted to CMAC operations, and WP makes even ICUSE refuse to rewrite it. The cheapest attack becomes physically opening the chip — a fundamentally different (and much more expensive) class of adversary.

4 • Knowledge check

Q1 • Where does the key value inside KEY_3 come from?

the ICUSE true random generator

a KDF from the SECRET_KEY

a factory tool over UDS

constants compiled into the application

SecSB_GenerateRamKey → SecLib_RandomGenerate (TRNG). The value is generated, not derived; K1–K4 are derived, but they only protect the transport of M2, not the key's origin.

Q2 • In a 64-byte secure block, where does the CMAC sit?

CMAC first, then payload bytes

payload first, then CMAC bytes

CMAC sandwiched between payload halves

CMAC at payload middle offset

data_protection.c:48,120 — the MAC occupies data[0..15], the CMAC is computed over data[16..63]. 16 bytes of MAC + 48 bytes of payload.

Q3 • What do the KU and WP flags jointly enforce on KEY_3?

CMAC only, and frozen forever

encryption only, and frozen forever

CMAC only, and freely rewritable

encryption only, and freely rewritable

KU=1 forbids ECB/CBC and permits CMAC generate/verify (manual Table 4.2); WP is a one-way latch — the slot can never be rewritten or erased (manual §4.3).

Q4 • CMAC verification fails on read while secure boot is enabled. What happens?

payload is zeroed and error returned

backup block is accepted without verification

payload is served with a warning

key slot is erased and regenerated

data_protection.c:79–86 — after secure boot is enabled there is no fallback: the whole block is memset to 0 and the error is returned. Backup fallback only exists pre-enable and under FLS_DRIVER.

5 • Read next / practice

PRIMARY SOURCE

ICUSE User Manual (docs/rh850_f1km_icuse_usermanual.pdf), **§4.4 "How to Register Keys", pages 43–45** for the M1–M5/KDF registration protocol, and §4.3 Table 4.2 for the flags. For the CMAC primitive itself, the HIS SHE specification (linked in RESOURCES.md) shows why AES-CMAC is the integrity workhorse.

- Companion fact sheet: DATASTORE_KEY & data-at-rest integrity — block layout, flows, KDF formula, all anchors on one page.
- Practice: open `data_protection.c` and trace one write + one read of a secure block against the fact sheet. Then explain to yourself why `SecSB_EnableSecurity` creates DATASTORE_KEY *before* writing the SAM block.
- Review: Lesson 0001 (slot map) and the Key Slot Map — DATASTORE_KEY's row is the KEY_3 row.

YOUR TEACHER

Stuck on the M2 bit layout, the SHE KDF, or why backup fallback disappears after secure boot is enabled? Ask in the chat — no question is too small, and the quiz is much more valuable if you interrogate every wrong answer.

Citations: RH850/F1KM ICUSE User Manual R01UH0703EJ0100 Rev.1.00 §4.3–4.4, §5.6.7 · project `security/secure_boot/secure_boot.c` , `security/crypto_lib/{crypto_library.c,key_manager.c}` , `security/hta_storage/{data_protection.c,nvm_driver.c}` .
Course: 0001 — where do the keys live? · reference — key slot map · reference — datastore integrity